

PRESSTO: PREdictive Switching for STOchastic Control of Gene Regulatory Networks

User's Guide (Version 1.0)

Christian Fernández-Pérez*

Manuel Pájaro[†]

Irene Otero-Muras*



Universidade de Vigo

Contents

1	General Overview and Requirements	2
1.1	Software Requirements and Compatibility	2
2	Theoretical and Numerical Foundations	2
2.1	The Multidimensional PIDE Model for Stochastic GRNs	2
2.2	The Semi-Lagrangian Numerical Approximation Scheme	4
2.3	Predictive-Switching Control (PSC) Framework	5
3	Software Architecture and Data Structures	6
3.1	Installation	6
3.2	Directory Layout and File Organization	7
3.3	Defining a Custom Case Study	7
3.3.1	The <code>grn</code> Struct: Biological and Kinetic Parameters	8
3.3.2	The <code>meshxt</code> Struct: Spatiotemporal Mesh and Control Window	9
3.3.3	The <code>ic</code> Struct: Population Initial Density Configuration	9
3.3.4	The <code>psc</code> Struct: Optimizer Settings and Cost Functionals	9
4	Computational Tutorials and Validation Cases	10
4.1	Case 1: One-gene regulatory systems	10
4.1.1	Example_1D: Mean Regulation	10
4.2	Case 2: Two-gene regulatory systems	14
4.2.1	Example_2D: Bimodality Repression	14
4.2.2	Example_2D_w20: Horizon Expansion ($w = 20$)	19
4.2.3	Example_2D_w100: Large Control Horizon ($w = 100$)	21
4.2.4	Example_2D_KLdiv: Kullback-Leibler Divergence Minimization	21
4.2.5	Example_2D_1I: Underactuated Mode Selection	24
4.3	Case 3: Three-gene regulatory systems	25
4.3.1	Example_3D: Symmetry Forcing	25

*Institute for Integrative Systems Biology (I2SysBio), Spanish National Research Council (CSIC), 46015 Valencia, Spain.

[†]Department of Mathematics, Universidade de Vigo, Campus Ourense 32004; CITMAga, Santiago de Compostela, Spain.

1 General Overview and Requirements

- **Repository / URL:** <https://github.com/ChristianFdz9/PRESSTO>
- **Authors:** Christian Fernández-Pérez, Manuel Pájaro, Irene Otero-Muras.
- **Contact Email:** christian.fernandez.perez@csic.es
- **Current Release:** 1.0
- **Distribution License:**
- **Copyright:** CSIC, Spanish National Research Council.

1.1 Software Requirements and Compatibility

The PRESSTO framework is cross-platform, supporting standard MATLAB installations across Windows, Linux, and macOS environments. The source code has been verified and benchmarked using MATLAB versions R2025b and subsequent releases.

If a CUDA-enabled NVIDIA GPU is detected (`gpuDeviceCount > 0`), the toolbox automatically accelerates the PIDE integration and control loop by executing operations via `gpuArray` structures without user intervention.

2 Theoretical and Numerical Foundations

PRESSTO models stochastic gene regulatory networks (GRNs) through a multidimensional partial integro-differential equation (PIDE) framework capable of describing transcriptional bursting, protein degradation, regulatory interactions, and external control actions. The formulation is based on the stochastic gene expression model proposed in [Pájaro et al., 2017] and serves as the mathematical foundation for all simulation and control algorithms implemented in the toolbox.

2.1 The Multidimensional PIDE Model for Stochastic GRNs

Consider a GRN composed of n genes, where X_i denotes the concentration of the i -th protein. The biological processes underlying the model, including transcription, translation, and regulatory feedback mechanisms, are schematically illustrated in Fig. 1.

Under the common assumption of transcriptional bursting, where mRNA degradation occurs on a significantly faster timescale than protein degradation ($\gamma_m^i \gg \gamma_x^i$), the temporal evolution of the probability density function (PDF) $p(t, \mathbf{x})$ is governed by the multidimensional PIDE

$$\frac{\partial p(t, \mathbf{x})}{\partial t} = \sum_{i=1}^n \frac{\partial}{\partial x_i} [\gamma_x^i x_i p(t, \mathbf{x})] + \sum_{i=1}^n k_m^i \int_0^{x_i} \beta_i(x_i - y_i) c_{i,\mathbf{u}}(\mathbf{y}_i) p(t, \mathbf{y}_i) dy_i, \quad (1)$$

The first term on the right-hand side accounts for the deterministic degradation of proteins, while the integral term describes stochastic burst-like production events. The burst kernel is given by

$$\beta_i(x_i - y_i) = \frac{1}{b_i} \exp\left(-\frac{x_i - y_i}{b_i}\right) - \delta(x_i - y_i), \quad (2)$$

where $b_i = k_m^i / \gamma_m^i$ is the mean burst size and $\delta(\cdot)$ denotes the Dirac delta distribution. The auxiliary vector $\mathbf{y}_i$ is defined such that $(\mathbf{y}_i)_j = x_j$ for $j \neq i$ and $(\mathbf{y}_i)_i = y_i$.

Gene regulation is represented through the activation probability function $c_{i,\mathbf{u}}(\mathbf{x}) \in [0, 1]$, which depends on both the system state and the external control input vector

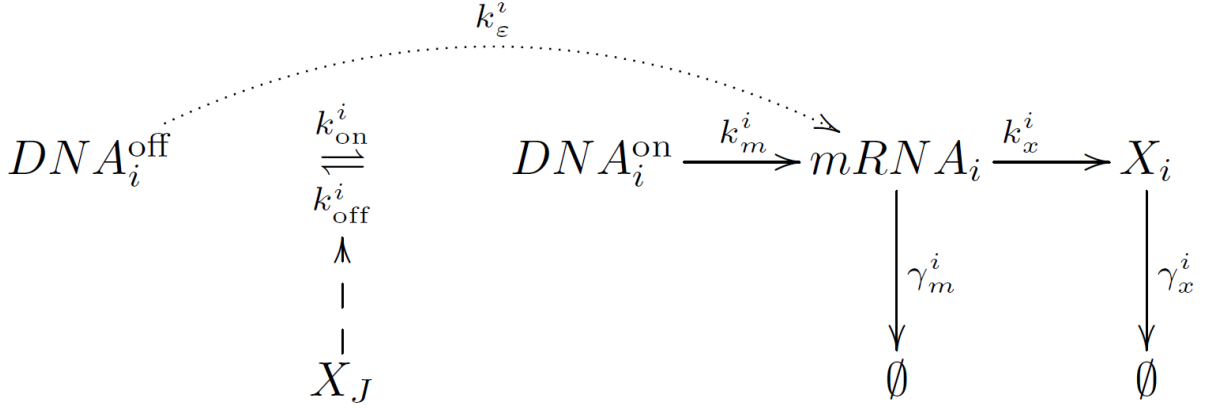


Figure 1: Schematic representation of the gene regulatory network considered in PRESSTO. $DNA_i^{\text{on/off}}$ denote active and inactive promoter states with transition rates $k_{\text{on/off}}^i$ regulated by protein X_j . Parameters k_m^i , k_x^i , and $\gamma_{m/x}^i$ correspond to transcription, translation, and degradation rates, respectively, while k_ϵ^i accounts for transcriptional leakage.

$$\mathbf{u}(t) = [I_1(t), \dots, I_n(t)]. \quad (3)$$

For Hill-type regulatory interactions, the contribution of protein X_j to gene i is modeled as

$$\rho_{ij}(x_j) = \frac{x_j^{H_{ij}}}{x_j^{H_{ij}} + \theta_{ij}^{H_{ij}}}, \quad (4)$$

where H_{ij} and K_{ij} denote the Hill coefficient and Hill constant, respectively.

External regulation is introduced through inducer molecules whose effect is described by the modulation function

$$F_i(I_i) = \left[1 + \left(\frac{I_i}{\theta_i} \right)^{\mu_i} \right]^{-1}, \quad (5)$$

where θ_i represents the half-saturation concentration and μ_i determines the steepness of the inducer response.

For repressive interactions, the corresponding controlled regulatory function becomes

$$\bar{\rho}_{ij,\mathbf{u}}(x_j) = \frac{K_{ij}^{H_{ij}}}{K_{ij}^{H_{ij}} + x_j^{H_{ij}} F_i(I_i)}. \quad (6)$$

To account for basal gene expression, the model incorporates a leakage factor $\epsilon_i = k_\epsilon^i/k_m^i$, which yields the effective regulatory function

$$c_{i,\mathbf{u}}(\mathbf{x}) = \bar{\rho}_{ij,\mathbf{u}}(x_j) + \epsilon_i(1 - \bar{\rho}_{ij,\mathbf{u}}(x_j)). \quad (7)$$

This multidimensional PIDE constitutes the core mathematical model implemented in PRESSTO. User-defined GRNs are translated into the corresponding regulatory functions and kinetic parameters, allowing the toolbox to simulate probability density evolution and to apply the control methodologies described in the following sections.

2.2 The Semi-Lagrangian Numerical Approximation Scheme

The multidimensional PIDE introduced in the previous section is solved in PRESSTO using a semi-Lagrangian numerical scheme based on the methodology presented in [Pájaro et al., 2017, 2018]. This approach combines characteristic tracing for the transport term with numerical quadrature for the integral contribution, providing an efficient framework for the simulation of stochastic gene regulatory networks.

The method begins by introducing the material derivative associated with the degradation vector field:

$$\mathbf{\Gamma}_x(\mathbf{x}) = (\gamma_x^1 x_1, \dots, \gamma_x^n x_n), \quad (8)$$

such that the total derivative is defined as:

$$\frac{Dp}{Dt} = \frac{\partial p}{\partial t} + \mathbf{\Gamma}_x(\mathbf{x}) \cdot \nabla_{\mathbf{x}} p, \quad (9)$$

where $\nabla_{\mathbf{x}}$ denotes the gradient operator with respect to the state vector $\mathbf{x}$.

For the temporal discretization, a uniform time mesh $\{t_k\}_{k=0}^{N_t}$ is defined over the interval $[0, T]$ with a constant time step:

$$\Delta t_k = \frac{T}{N_t}. \quad (10)$$

The material derivative evaluated at $(t_{k+1}, \mathbf{x})$ is approximated by the backward Euler difference:

$$\frac{Dp}{Dt}(t_{k+1}, \mathbf{x}) \approx \frac{p(t_{k+1}, \mathbf{x}) - p(t_k, \bar{\mathbf{x}}_k)}{\Delta t}, \quad (11)$$

where $\bar{\mathbf{x}}_k$ denotes the departure point obtained by tracing the characteristic curve backward in time from $\mathbf{x}$.

The characteristic trajectories are computed from the system of ordinary differential equations:

$$\frac{\partial \chi}{\partial \tau}(\mathbf{x}, t_{k+1}; \tau) = -\mathbf{\Gamma}_x(\chi(\mathbf{x}, t_{k+1}; \tau)), \quad (12)$$

subject to the final condition:

$$\chi(\mathbf{x}, t_{k+1}; t_{k+1}) = \mathbf{x}. \quad (13)$$

The departure point is therefore given by:

$$\bar{\mathbf{x}}_k = \chi(\mathbf{x}, t_{k+1}; t_k). \quad (14)$$

Combining the characteristic approximation with the numerical evaluation of the integral term yields the explicit update formula implemented in PRESSTO:

$$p_{k+1}(\mathbf{x}) = \frac{p_k(\bar{\mathbf{x}}_k) + \Delta t I(p_k)(t_k, \mathbf{x})}{1 + \Delta t_k [\sum_{i=1}^n k_m^i c_{i,u}(\mathbf{x}) + (\nabla \cdot \mathbf{\Gamma}_x)(\mathbf{x})]}, \quad (15)$$

where $I(p_k)(t_k, \mathbf{x})$ denotes the numerical approximation of the integral production term and $(\nabla \cdot \mathbf{\Gamma}_x)(\mathbf{x})$ is the divergence of the degradation field.

The computational domain is discretized using a uniform Cartesian mesh. For each protein concentration variable x_i , the interval $[0, L_i]$ is partitioned into N_i subintervals of length:

$$h_i = \frac{L_i}{N_i}, \quad (16)$$

yielding the grid nodes:

$$(q_1 h_1, \dots, q_n h_n), \quad q_i = 0, \dots, N_i. \quad (17)$$

The integral contribution $I(p_k)$ is evaluated using a multidimensional composite trapezoidal quadrature rule. Since the departure point $\bar{\mathbf{x}}_k$ generally does not coincide with a grid node,

interpolation is required to estimate $p_k(\bar{\mathbf{x}}_k)$. Following the implementation proposed in SELANSI [Pájaro et al., 2018], PRESSTO employs multilinear interpolation, providing a suitable compromise between computational efficiency and numerical accuracy.

This semi-Lagrangian framework constitutes the numerical engine used throughout the toolbox for solving the multidimensional PIDE model and serves as the basis for all simulation and control routines available in PRESSTO.

2.3 Predictive-Switching Control (PSC) Framework

Predictive-Switching Control (PSC) is the strategy implemented in PRESSTO to steer stochastic gene regulatory networks toward user-defined target objectives. The methodology operates directly on the population probability density function predicted by the multidimensional PIDE model, determining at each decision instant the specific inducer configuration that optimizes a prescribed cost functional.

PSC belongs to the family of finite-set model predictive control methods. Instead of computing continuous-valued control actions, the controller selects the optimal input from a finite set of admissible discrete alternatives. At every control update, the future evolution of the probability distribution is predicted under all candidate input configurations, and the configuration that best satisfies the control objective is selected.

The numerical integration of the underlying PIDE is performed on the fine temporal grid $\{t_k\}_{k=0}^K$ introduced previously. In contrast, PSC decisions are updated on a coarser switching grid $\{t_m\}_{m=0}^M$. The control action is constrained to remain constant during an actuation window of duration:

$$\Delta t_m = w \cdot \Delta t_k, \quad (18)$$

where $w \in \mathbb{N}_{\geq 1}$ is a user-defined switching parameter. The total control horizon is therefore partitioned as:

$$\mathcal{T} = \bigcup_{m=0}^{M-1} (t_m, t_{m+1}], \quad (19)$$

with the switching discrete times defined by:

$$t_m = m \cdot \Delta t_m. \quad (20)$$

The parameter w directly determines the frequency at which the inducer configuration is updated; smaller values increase the temporal resolution of the controller, enabling rapid reactions to transient variations in the PDF, whereas larger values reduce the number of optimization steps and decrease the overall computational cost.

Each chemical inducer operates according to a binary switching logic and can assume one of two distinct functional states:

- OFF: The inducer concentration is equal to zero.
- ON: The inducer concentration is fixed at a specific saturation level.

For each inducer I_i , the saturation level κ_i is computed from a user-defined suppression threshold $\alpha_i \in (0, 1)$ such that:

$$\bar{\rho}_{ij,\mathbf{u}}(L_j, \kappa_i) = 1 - \alpha_i. \quad (21)$$

where L_j denotes the upper bound of the computational domain for the regulating protein X_j . This ensures that the specified suppression level α_i is strictly guaranteed even under full repressor abundance. The resulting saturation vector:

$$\boldsymbol{\kappa} = [\kappa_1, \dots, \kappa_n]^T \quad (22)$$

defines the maximum operating level of every external input.

Since each of the n inducers can be independently toggled ON or OFF, the controller evaluates a total of 2^n admissible configurations. These combinations are structured via a binary matrix $S \in \{0, 1\}^{2^n \times n}$, where each row S_r represents a unique switching state. The physical inducer vector associated with configuration r is computed via the element-wise Hadamard product:

$$\mathbf{u}^{(r)} = \boldsymbol{\kappa} \odot S_r^T, \quad r = 1, \dots, 2^n, \quad (23)$$

yielding the finite admissible control set:

$$\mathcal{U} = \left\{ \mathbf{u}^{(r)} \right\}_{r=1}^{2^n}. \quad (24)$$

At each switching instant t_m , the PSC framework uses the current probability distribution $p(t_m, \mathbf{x})$ as the initial condition for a parallel set of predictive simulations. For every admissible configuration $\mathbf{u}^{(r)} \in \mathcal{U}$, the PIDE model is integrated over the single control interval $(t_m, t_{m+1}]$, producing a set of candidate predicted distributions:

$$\{p_{\mathbf{u}^{(r)}}(t_{m+1}, \mathbf{x})\}_{r=1}^{2^n}. \quad (25)$$

A prescribed cost functional $J(\cdot)$ is then evaluated for each prediction. Depending on the design setup, the optimal inducer configuration $\mathbf{u}_m$ is obtained by solving either a maximization or minimization problem:

$$\mathbf{u}_m = \arg \max_{\mathbf{u}^{(r)} \in \mathcal{U}} J(p_{\mathbf{u}^{(r)}}(t_{m+1}, \mathbf{x})), \quad (26)$$

or equivalently using $\arg \min$ for minimization targets.

The selected configuration $\mathbf{u}_m$ is applied fixed during the entire interval $(t_m, t_{m+1}]$. The resulting end-point distribution then serves as the initial state for the next decision step, repeating the receding-horizon process until the final simulation time is reached. The sequence:

$$\mathbf{u}^* = (\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{M-1}) \quad (27)$$

defines the complete piecewise-constant control trajectory generated by the system.

A complete theoretical formulation of the Predictive-Switching Control framework, including stability analysis and detailed derivations, can be found in Fernández et al. [2026].

3 Software Architecture and Data Structures

PRESSTO (PREdictive Switching for STOchastic Control of Gene Regulatory Networks) is an integrated MATLAB toolbox designed to simulate the multi-dimensional probability landscape of gene networks and compute optimal piecewise-constant control policies. The software operates by coupling a high-performance semi-Lagrangian solver with a model predictive control loop, enabling users to orchestrate population-level phenotypes without managing the underlying numerical optimization mechanics.

3.1 Installation

To deploy the toolbox, simply place the PRESSTO folder into your chosen working directory. All toolbox functions and simulation scripts must be executed directly from the root directory (PRESSTO) to ensure correct relative path resolution.

3.2 Directory Layout and File Organization

The framework enforces a highly structured directory layout to decouple biological design from execution logic and computational results. The workspace is organized into three primary folders alongside four core engine scripts:

- **DOCUMENTATION/**: Contains the user guide, technical specifications, and reference manuals.
- **EXPERIMENTS/**: Consists of user-defined MATLAB functions describing the target biological study case (e.g., `case_of_study_1.m`). It acts as the exclusive interface where network parameters, meshes, and cost functions are specified.
- **SIMULATIONS/**: Automatically created by the software engine. For each study case executed, a subdirectory with the exact name of the file in **EXPERIMENTS** is generated to store two data files by default: `config.mat` and `Results.mat`.



Figure 2: Expected organization of folders and main scripts in the MATLAB Current Folder browser.

At the root level, the operational workflow is governed by four core functions:

1. **PRESSTO_main.m**: The main control script. The user initiates execution solely from this script by supplying the case study name, for instance: `PRESSTO_main('case_of_study_1')`. It reads the setup, pre-allocates memory, and runs the main loop.
2. **PSC.m**: The control engine. It checks the structural validity of the data structures, evaluates the configuration tree, analyzes the cost functional, and schedules predictions.
3. **PIDE.m**: The numerical engine. It solves the multidimensional partial integro-differential equations over a specific time window utilizing the explicit semi-Lagrangian scheme.
4. **PLOT.m**: The visualization script. It automatically generates probability density plots and control trajectory layouts once execution terminates. Note: If you wish to re-plot or customize the figures after the simulation has finished, you can re-run this script at any time from the MATLAB Command Window by executing:

```
>> PLOT('case_of_study_1');
```

This eliminates the need to re-integrate the system, as it directly fetches the saved data from the corresponding **SIMULATIONS** folder.

3.3 Defining a Custom Case Study

The entire workflow of PRESSTO is designed around a strict principle of isolation: the user only needs to interact with a single configuration file located within the **EXPERIMENTS/** directory. This file centralizes all information regarding the biological system, spatial grids, and control performance parameters. The software engine parses this file into four core sub-structures nested inside a master `config` variable.

3.3.1 The grn Struct: Biological and Kinetic Parameters

The **grn** structure translates the fundamental biochemical reactions that govern gene regulatory networks into programmable functions. Under the steady-state assumption for fast mRNA dynamics ($\gamma_m^i \gg \gamma_x^i$), the model implicitly accounts for the four primary reactions defining the life cycle of each protein X_i :

1. Transcription: $\emptyset \xrightarrow{k_m^i c_{i,u}(\mathbf{x})} \text{mRNA}_i$
2. Translation: $\text{mRNA}_i \xrightarrow{k_x^i} \text{mRNA}_i + X_i$
3. mRNA Degradation: $\text{mRNA}_i \xrightarrow{\gamma_m^i} \emptyset$
4. Protein Degradation: $X_i \xrightarrow{\gamma_x^i} \emptyset$

These coupled mechanisms collapse into stochastic burst-like production events of mean size $b_i = k_x^i / \gamma_m^i$ modulated by the effective activation function $c_{i,u}(\mathbf{x})$. To set up this network, the configuration script requires the definition of the following fields:

- **grn.n_gene**: Scalar specifying the total number of genes (n) defining the state space dimensionality.
- **grn.k_m**: $1 \times n$ vector of maximum transcription rates (k_m^i) for each gene.
- **grn.k_x**: $1 \times n$ vector of translation rates (k_x^i).
- **grn.g_m**: $1 \times n$ vector of mRNA degradation rates (γ_m^i), defining the scale of the mean burst size b_i .
- **grn.g_x**: $1 \times n$ vector of protein degradation rates (γ_x^i), driving the deterministic velocity field $\Gamma_x(\mathbf{x})$.
- **grn.eps**: $1 \times n$ vector of leakage factors ($\varepsilon_i = k_\varepsilon^i / k_m^i$) capturing basal expression.
- **grn.F**: Cell array containing the functional definitions of the chemical inducer responses $F_i(I_i)$ following Eq. (5). It is parameterized by half-saturation vectors (**grn.theta**) and Hill coefficients (**grn.mu**), evaluating as:

$$\text{grn.F}\{i\}(I_i) = \left[1 + \left(\frac{I_i}{\text{grn.theta}(i)} \right)^{\text{grn.mu}(i)} \right]^{-1}. \quad (28)$$

- **grn.rho**: Cell array of functions implementing the cross-regulatory Hill terms that capture network activation/repression topologies. For a repressive interaction from protein X_j to gene i modulated by an inducer, it computes Eq. (6) using the full state and input structures:

$$\text{grn.rho}\{i\}(\mathbf{x}, \mathbf{u}) = \frac{\text{grn.K}(i)^{\text{grn.H}(i)}}{\text{grn.K}(i)^{\text{grn.H}(i)} + \mathbf{x}\{j\}^{\text{grn.H}(i)} \cdot \text{grn.F}\{i\}(u_i)}. \quad (29)$$

Note: While the theoretical formulation models the regulation layout using full topological dimensions (mapping the interaction from repressor j to target gene i via $\rho_{ij}, K_{ij}, H_{ij}$), the toolbox implementation simplifies this by assigning a single index **grn.rho**{i} to each gene. The user simply writes the local cross-regulation terms inside the function handle using vector indexing (**grn.K**(i), **grn.H**(i)) and addressing the specific regulator via the cell element $\mathbf{x}\{j\}$, optimizing both script execution and readability.

- **grn.c**: Cell array binding the joint contribution of active regulation and basal leakage (ε_i) to evaluate the net core activation probability function $c_{i,u}(\mathbf{x})$ introduced in Eq. (7). This function directly feeds the integral operator of the PIDE solver:

$$\text{grn.c}\{i\}(\mathbf{x}, \mathbf{u}) = \text{grn.rho}\{i\}(\mathbf{x}, \mathbf{u}) + \text{grn.eps}(i) \cdot (1 - \text{grn.rho}\{i\}(\mathbf{x}, \mathbf{u})). \quad (30)$$

3.3.2 The `meshxt` Struct: Spatiotemporal Mesh and Control Window

The `meshxt` structure configures the spatial grids and temporal steps required to resolve the multi-dimensional probability evolution.

- `meshxt.Prot_mesh`: An $n \times 3$ matrix. Each row i defines the boundaries for protein X_i via the array format: $[x_{i,\min}, x_{i,\max}, N_i]$, establishing the grid node length:

$$\Delta x_i = (x_{i,\max} - x_{i,\min})/N_i. \quad (31)$$

- `meshxt.Time_mesh`: A 1×3 vector structured as $[t_0, t_{\text{end}}, N_t]$, defining the boundary limits of the time horizon and the total integration steps. The baseline PIDE integration step is

$$\Delta t_k = (t_{\text{end}} - t_0)/N_t. \quad (32)$$

- `meshxt.w`: Integer scalar $w \in \mathbb{N}_{\geq 1}$ representing the size of the actuation window.

The switching parameter w defines the bridge between the fine numerical grid $\{t_k\}$ used by the semi-Lagrangian solver in `PIDE.m` and the coarse control grid $\{t_m\}$ used by the predictive optimization loop in `PSC.m`. The relationship is illustrated in Fig. 3.

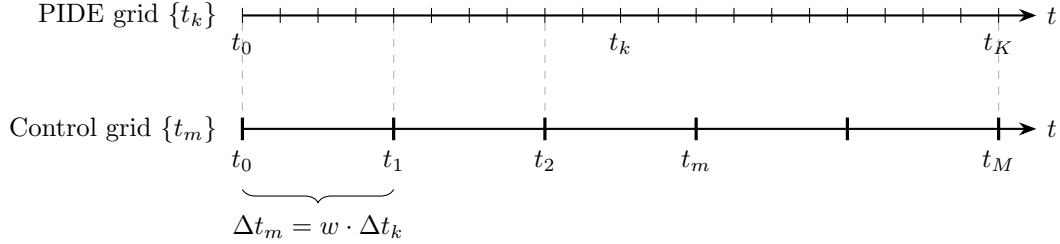


Figure 3: Relationship between the fine numerical grid $\{t_k\}$ used by the PIDE solver and the coarse switching grid $\{t_m\}$ used by the controller. Each actuation window Δt_m contains w integration steps.

3.3.3 The `ic` Struct: Population Initial Density Configuration

The `ic` structure governs how the probability density function is initialized at $t = t_0$ across the state space mesh.

- `ic.mode`: Functional flag toggle.
 - `ic.mode = 0`: Configures an analytical multivariate Gaussian distribution using the mean vector `ic.mean` and variance arrays `ic.var`.
 - `ic.mode = 1`: Instructs the engine to load a custom numerical distribution from an external data file. This enables initializing the control sequence from pre-computed experimental data or steady-state profiles.

3.3.4 The `psc` Struct: Optimizer Settings and Cost Functionals

The operational execution of the Predictive-Switching Control algorithm is parameterized by the fields of the `psc` structure:

- `psc.alpha`: The suppression threshold $\alpha_i \in (0,1)$ utilized to internally compute the physical saturation bounds κ of the system according to Eq. (21):

$$\bar{\rho}_{ij,\mathbf{u}}(x_{i,\max}, \kappa_i) = 1 - \alpha_i. \quad (33)$$

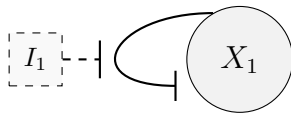
- `psc.cost_f`: A MATLAB function handle evaluating the performance metric $J(p(t, \mathbf{x}))$. It processes the multi-dimensional PDF array and returns a scalar value.
- `psc.opt`: Binary optimization selector. Setting `psc.opt = 1` instructs the engine to look for maximization objectives ($\arg \max$), whereas `psc.opt = 0` triggers minimization routines ($\arg \min$).

4 Computational Tutorials and Validation Cases

In this section, we illustrate the performance of the Predictive-Switching Control (PSC) algorithm using the PRESSTO toolbox through a number of benchmark examples. These cases are implemented by defining the corresponding configuration functions within the `EXPERIMENTS` folder. Once defined, any example can be directly simulated using the main routine `PRESSTO_main` and visualized with the `PLOT` function.

4.1 Case 1: One-gene regulatory systems

Low-dimensional architectures ($n = 1$) are analyzed here to validate the core PSC algorithm implemented in the PRESSTO framework under isolated regulatory loops. The considered network consists of a single self-repressing gene under the action of an external inducer, as shown in Fig. 4, together with the kinetic parameters used throughout this example.



Parameter	Value
Transcription rate (k_m)	20
Translation rate (k_x)	40
mRNA degradation rate (γ_m)	15
Protein degradation rate (γ_x)	1
Leakage rate (ε)	0.15

Figure 4: Network topology and biochemical parameter identification for the 1D self-regulated gene case study with external induction (I_1).

4.1.1 Example_1D: Mean Regulation

This case study describes the control of a self-regulating gene network under the influence of an external inducer profile. The system models the coupled stochastic dynamics of mRNA and its corresponding protein by incorporating fundamental biochemical processes: transcription regulated via a negative feedback loop, translation, mRNA and protein degradation, and basal expression or leakage rates. In this context, the chemical inducer I acts as an external inhibitor of gene transcription, effectively modulating the binding affinity of the repressor to the promoter region.

Without external intervention, the open-loop system naturally converges toward a stationary probability distribution defining its unforced steady state. Under these basal conditions, the population profile exhibits a clear attraction basin where the mean protein copy number is $\bar{X} \approx 38$, as illustrated in Figure 5. The primary objective of the Predictive-Switching Control (PSC) algorithm is to dynamically manipulate the time-varying inducer profile. By calculating optimal switching times, the controller aims to shift the population's probability mass across the state space, effectively driving the mean protein count from its basal level of 38 to a significantly higher target value of $\bar{X}^* = 54$.

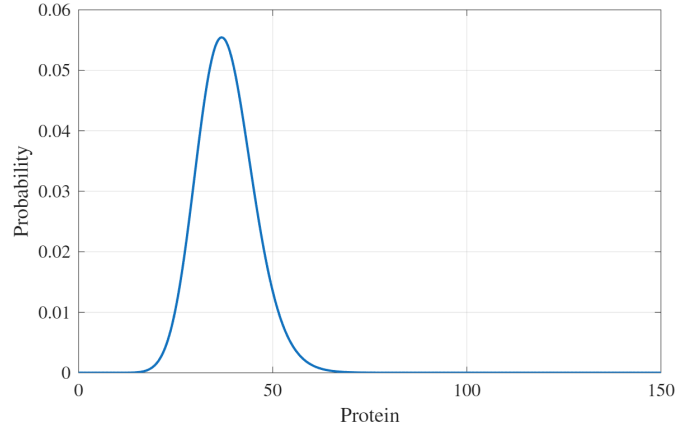


Figure 5: Steady-state protein distribution ($\bar{X} \approx 38$) before control application.

The configuration Script. 1 (Example_1D.m) is already provided inside the EXPERIMENTS/ folder. It's not necessary to copy or create it. The script structures the network kinetics, simulation grids, and control parameters as follows:

Script 1: Complete configuration for the 1D GRN example (Example_1D.m).

```

1 function config = Example_1D()
2 %% ===== OUTPUT FOLDER =====
3 config.name = mfilename;
4
5 %% ===== GENE REGULATORY NETWORK =====
6 % Definition of network size and kinetic rates
7 grn.n_gene = 1;      % Number of genes
8 grn.k_m = 20;        % Transcription rate
9 grn.k_x = 40;        % Translation rate
10 grn.g_m = 15;        % mRNA degradation rate
11 grn.g_x = 1;         % Protein degradation rate
12 grn.eps = 0.15;      % Leakage rate
13
14 % --- Inducer functions (F) ---
15 % Effects of the external inducer profile on transcription
16 grn.F = cell(1, grn.n_gene);
17 grn.theta = 0.1;
18 grn.mu = 4;
19 grn.F{1} = @(I) 1 ./ (1 + (I ./ grn.theta(1)).^grn.mu(1));
20
21 % --- Regulatory functions (rho) ---
22 % Non-linear Hill-type gene inhibition logic
23 grn.rho = cell(1, grn.n_gene);
24 grn.K = 45;
25 grn.H = 4;
26 grn.rho{1} = @(x,u) grn.K(1)^grn.H(1) ./ ...
27     (grn.K(1)^grn.H(1) + (x{1}.^grn.H(1)).*(grn.F{1}(u(1))));
28
29 % --- Net input functions (c) ---
30 % Combines active regulation and transcription leakage
31 grn.c = cell(1, grn.n_gene);
32 grn.c{1} = @(x,u) grn.rho{1}(x,u) + grn.eps(1)*(1-grn.rho{1}(x,u));
33
34 config.grn = grn;
35
36 %% ===== MESH =====

```

```

37 % Spatial grid (protein counts) and temporal simulation mesh
38 meshxt.Prot_mesh = [0 150 600]; % [min, max, num. of spatial intervals]
39 meshxt.Time_mesh = [0 2 400]; % [t0, t_end, number of time intervals]
40 meshxt.w = 10; % Control horizon window size
41
42 config.meshxt = meshxt;
43
44 %% ===== INITIAL CONDITION =====
45 ic.mode = 0; % 0 = Analytical Gaussian, 1 = Load external .mat
46     distribution
47 switch ic.mode
48     case 0
49         ic.mean = 100; % Initial population mean
50         ic.var = 5; % Initial population variance
51     end
52 config.ic = ic;
53
54 %% ===== PSC PARAMETERS =====
55 % Activation level = 1 - alpha_i (one per inducer)
56 psc.alpha = 0.05;
57
58 % Cost function
59 target_mean = 54;
60 x = linspace(meshxt.Prot_mesh(1), meshxt.Prot_mesh(2), meshxt.Prot_mesh
61     (3) + 1);
62 psc.cost_f = @(PX) (sum(x(:) .* PX(:)) / sum(PX(:)) - target_mean).^2;
63
64 % Optimization mode: 0 = Minimize, 1 = Maximize
65 psc.opt = 0;
66
67 config.psc = psc;
68 end

```

The main operational parameters governing the Predictive-Switching Control framework can be adjusted directly within the script via the following configuration fields:

- **Control Horizon (meshxt.w):** Set to 10 steps, specifying the length of the look-ahead prediction window for the optimization process.

```
meshxt.w = 10; % Control horizon window size (steps)
```

- **Initial Condition (ic.mode, ic.mean, ic.var):** Configured to initialize the system state profile from a tight Gaussian distribution $x_0 \sim \mathcal{N}(100, 5)$ (ic.mode=0).

```
ic.mode = 0; % 0 = Analytical Gaussian
ic.mean = 100; % Initial population mean
ic.var = 5; % Initial population variance
```

- **Boundary Suppression Threshold (psc.alpha):** Defines the suppression tolerance $\alpha_1 \in (0, 1)$ used to compute the saturation value κ_1 of the inducer, establishing the exact input concentration required to enforce the target repression probability at the grid boundary.

```
psc.alpha = 0.05; % Suppression threshold for physical bounds
```

- **Cost Function (psc.cost_f, psc.opt):** Formulated to track and minimize the distance between the evolving population mean and the setpoint $\bar{X}^* = 54$ via the quadratic expression. Setting `psc.opt = 0` configures the algorithm to run in minimization mode:

$$J(P_X) = \left(\frac{\sum x_i P_X(x_i)}{\sum P_X(x_i)} - X_d \right)^2 \quad (34)$$

```
target_mean = 54;
x = linspace(meshxt.Prot_mesh(1), meshxt.Prot_mesh(2), meshxt.
    Prot_mesh(3) + 1);
psc.cost_f = @(PX) (sum(x(:) .* PX(:)) / sum(PX(:)) - target_mean)
    .^2;
psc.opt = 0;                                     % 0 = Minimization mode
```

To run the optimization workspace, make sure you are in the toolbox root directory in MATLAB and execute the primary entry routine passing the experiment's name string:

```
PRESST0_main('Example_1D')
```

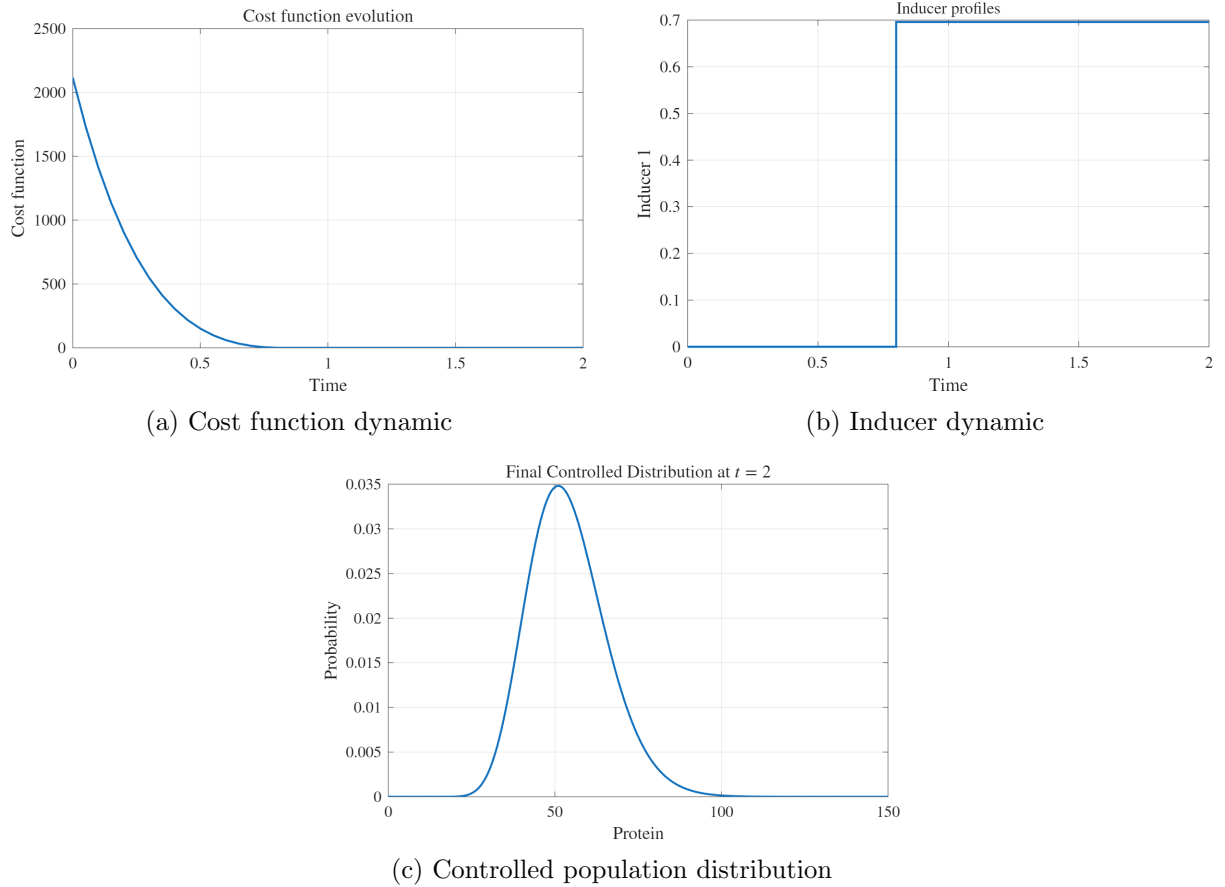


Figure 6: PSC simulation results for Example_1D showing tracking convergence and control actuation.

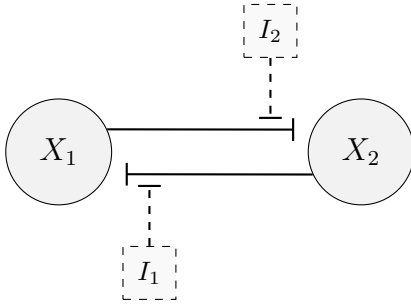
This command solves the numerical framework and renders the performance plots instantly. If you want to view or export the plots later on, you do not need to re-run the heavy simulation routines from scratch. Simply run the dedicated post-processing script from the root workspace:

```
PLOT('Example_1D')
```

The execution automatically generates the system performance results, which are illustrated and detailed in Fig. 6. The resulting layout demonstrates how the proposed PSC framework dynamically switches the chemical inducer profiles to drive the structural probability mass of the cell network efficiently toward its designated setpoint, minimizing the distance to the target value throughout the time horizon.

4.2 Case 2: Two-gene regulatory systems

Higher-dimensional architectures ($n = 2$) present complex behaviors, such as multi-stability and bimodality, where population density sheets split into distinct phenotypes. This section details the configuration and control of a two-gene network. The considered system consists of a classic genetic toggle switch with mutual repression between both nodes, as illustrated in Fig. 7, alongside its symmetric kinetic parameter setup.



Parameter	Value
Transcription rates (k_m)	[10, 10]
Translation rates (k_x)	[100, 100]
mRNA degradation rates (γ_m)	[10, 10]
Protein degradation rates (γ_x)	[1, 1]
Leakage rates (ε)	[0.1, 0.1]

Figure 7: Genetic toggle switch architecture showing corepression pathways and external inducer lines (I_1, I_2) countering protein-level suppression.

4.2.1 Example 2D: Bimodality Repression

This case study models a mutually repressive genetic toggle switch under the action of two independent external inducers. Due to strong non-linear cross-inhibition, the system naturally exhibits a bimodal steady-state population profile: the mass splits into two distinct phenotypes where either Gene 1 or Gene 2 dominates while repressing the other, as illustrated in Fig. 8. The objective of the Predictive-Switching Control (PSC) algorithm is to dynamically actuate both inducer profiles to override this natural bistability, suppressing the bimodal distribution by forcing each independent marginal density to concentrate its probability mass at the target coordinate $X_i^* = 48$ protein copies. Notably, this threshold corresponds exactly to the unstable valley of low probability density situated between the two modes of each unactuated marginal profile, representing a state that the open-loop system naturally evades.

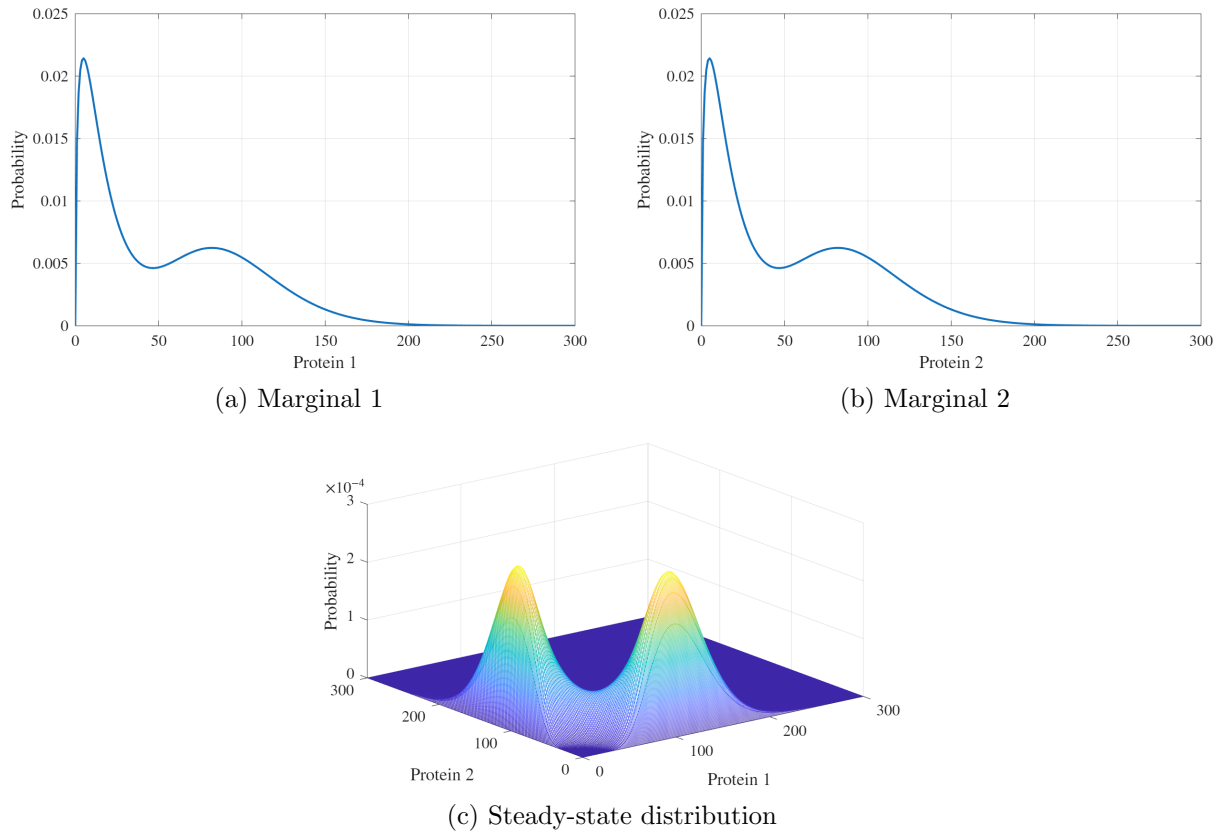


Figure 8: Steady-state bimodal distribution before applying control.

In the EXPERIMENTS/ directory, a ready-to-run copy is provided as `Example_2D.m`, with implementation details detailed in Script 2:

Script 2: Complete configuration for the 2D toggle switch case study (`Example_2D.m`).

```

1 function config = Example_2D()
2 %% ===== OUTPUT FOLDER =====
3 config.name = mfilename;
4
5 %% ===== GENE REGULATORY NETWORK =====
6 % Number of genes
7 grn.n_gene = 2;
8
9 % --- Rates ---
10 grn.k_m = [10 10];           % Transcription rates (k_m)
11 grn.k_x = [100 100];        % Translation rates (k_x)
12 grn.g_m = [10 10];          % mRNA degradation (g_m)
13 grn.g_x = [1 1];            % Protein degradation (g_x)
14 grn.eps = [0.1 0.1];        % Leakage rates (eps)
15
16 % --- Inducer functions (F) ---
17 grn.F = cell(1, grn.n_gene);
18 grn.theta = [0.1 0.1];
19 grn.mu     = [2 2];
20 grn.F{1} = @(I) 1 ./ (1 + (I ./ grn.theta(1)).^grn.mu(1));
21 grn.F{2} = @(I) 1 ./ (1 + (I ./ grn.theta(2)).^grn.mu(2));
22
23 % --- Regulatory functions (rho) ---
24 % Cross-inhibition functions between Gene 1 and Gene 2 (Hill-type)
25 grn.rho = cell(1, grn.n_gene);

```

```

26 grn.K = [40 40];
27 grn.H    = [4 4];
28 grn.rho{1} = @(x,u) grn.K(1)^grn.H(1) ./ ...
29     (grn.K(1)^grn.H(1) + (x{2}.^grn.H(1)) .* grn.F{1}(u(1)));
30 grn.rho{2} = @(x,u) grn.K(2)^grn.H(2) ./ ...
31     (grn.K(2)^grn.H(2) + (x{1}.^grn.H(2)) .* grn.F{2}(u(2)));
32
33 % --- Net input functions (c) ---
34 grn.c = cell(1, grn.n_gene);
35 grn.c{1} = @(x,u) grn.rho{1}(x,u) + grn.eps(1)*(1-grn.rho{1}(x,u));
36 grn.c{2} = @(x,u) grn.rho{2}(x,u) + grn.eps(2)*(1-grn.rho{2}(x,u));
37
38 config.grn = grn;
39
40 %% ===== MESH =====
41 % Spatial grid (protein counts) and temporal simulation mesh
42 meshxt.Prot_mesh = [0 300 300; 0 300 300]; % [min, max, number of
43     spatial intervals] per gene
44 meshxt.Time_mesh = [0 5 1000]; % [t0, t_end, number of time
45     intervals]
46 meshxt.w = 1; % Control horizon window
47     size
48
49 config.meshxt = meshxt;
50
51 %% ===== INITIAL CONDITION =====
52 ic.mode = 1; % 0 = Analytical Gaussian, 1 = Load external .mat
53     distribution
54 switch ic.mode
55     case 1
56         PX0_folder = fullfile(pwd, 'SIMULATIONS', config.name);
57         ic.PX0 = load(fullfile(PX0_folder, 'PX0.mat'), 'PX');
58     end
59
60 config.ic = ic;
61
62 %% ===== PSC PARAMETERS =====
63 % Activation level = 1 - alpha_i (one per inducer)
64 psc.alpha = [0.01 0.01];
65
66 % Cost function
67 psc.cost_f = @(PX) cost_function(PX);
68
69 % Optimization mode: 0 = Minimize, 1 = Maximize
70 psc.opt = 1;
71
72 config.psc = psc;
73 end
74
75 %% ===== LOCAL FUNCTIONS =====
76 function cost = cost_function(PX)
77     % 1. Normalize the joint probability distribution function (PDF)
78     max_val = max(PX(:));
79     PX_norm = PX ./ max_val;
80
81     % 2. Define the spatial grids for both genes
82     x{1} = linspace(0, 300, 301);
83     x{2} = linspace(0, 300, 301);

```



```

80
81 % 3. Compute marginal distributions
82 M_1 = sum(PX_norm, 2) / trapz(x{1}, sum(PX_norm, 2));
83 M_2 = sum(PX_norm, 1) / trapz(x{2}, sum(PX_norm, 1));
84
85 % 4. Peak-normalize the marginal distributions
86 [max_valM1, ~] = max(M_1(:));
87 M_1_norm = M_1 ./ max_valM1;
88
89 [max_valM2, ~] = max(M_2(:));
90 M_2_norm = M_2 ./ max_valM2;
91
92 % 5. Map target physical protein values to closest grid indexes
93 target_point = [48 48];
94 target_index = zeros(1, 2);
95
96 for i = 1:2
97     [~, target_index(i)] = min(abs(x{i} - target_point(i)));
98 end
99
100 % 6. Extract peak-normalized marginal probabilities
101 probM1 = M_1_norm(target_index(1));
102 probM2 = M_2_norm(target_index(2));
103
104 % 7. Net cost definition
105 cost = probM1 + probM2;
106 end

```

The core execution parameters regulating the Predictive-Switching Control engine are defined directly inside the script through the following configuration fields:

- **Control Horizon (meshxt.w):** Set to 1 step, reducing the prediction window to its minimum size to yield rapid control updates across the spatiotemporal grid and immediate reaction against the natural bifurcation.

```
meshxt.w = 1; % Control horizon window size
```

- **Initial Condition (ic.mode, ic.PX0):** Configured to 1 to instruct the engine to initialize the system density profile from a fully developed, pre-simulated uncontrolled bimodal distribution. In this case, the initial state matrix PX0.mat is retrieved directly from the Example_2D subfolder located within the SIMULATIONS/ directory.

```
ic.mode = 1; % 1 = Load external pre-computed bimodal
distribution
PX0_folder = fullfile(pwd, 'SIMULATIONS', config.name);
ic.PX0 = load(fullfile(PX0_folder, 'PX0.mat'), 'PX').PX;
```

- **Boundary Suppression Thresholds (psc.alpha):** Establishes the multi-variable vector $\alpha = [\alpha_1, \alpha_2]$ used to calculate the saturation value κ_i of each independent inducer, fixing the precise concentration levels needed to reach the targeted suppression probabilities across both gene dimensions.

```
psc.alpha = [0.01 0.01]; % Suppression thresholds per gene
```

- **Cost Function (psc.cost_f, psc.opt):** Formulated to isolate and maximize the peak-normalized marginal distributions ($M_{1,norm}$ and $M_{2,norm}$) directly at the under-expressed

coordinate $(X_1^*, X_2^*) = (48, 48)$, which sits in the unstable low-density valley. Since both marginals are peak-normalized to a maximum value of 1, the objective function reaches its absolute theoretical maximum at $J = 2$ when the mode of both independent marginal distributions is perfectly aligned with the target state. Setting `psc.opt = 1` configures the algorithm to run in maximization mode:

$$J(P_X) = M_{1,\text{norm}}(X_1^*) + M_{2,\text{norm}}(X_2^*) \quad (35)$$

```
psc.cost_f = @(PX) cost_function(PX); % Calls local peak-
    normalization routine
psc.opt = 1; % 1 = Maximization mode

function cost = cost_function(PX)
    % 1. Normalize the joint probability distribution function
    max_val = max(PX(:));
    PX_norm = PX ./ max_val;

    % 2. Define the spatial grids for both genes
    x{1} = linspace(0, 300, 301);
    x{2} = linspace(0, 300, 301);

    % 3. Compute marginal distributions
    M_1 = sum(PX_norm, 2) / trapz(x{1}, sum(PX_norm, 2));
    M_2 = sum(PX_norm, 1) / trapz(x{2}, sum(PX_norm, 1));

    % 4. Peak-normalize the marginal distributions
    [max_valM1, ~] = max(M_1(:));
    M_1_norm = M_1 ./ max_valM1;

    [max_valM2, ~] = max(M_2(:));
    M_2_norm = M_2 ./ max_valM2;

    % 5. Map target physical protein values to closest grid indexes
    target_point = [48 48];
    target_index = zeros(1, 2);

    for i = 1:2
        [~, target_index(i)] = min(abs(x{i} - target_point(i)));
    end

    % 6. Extract peak-normalized marginal probabilities
    probM1 = M_1_norm(target_index(1));
    probM2 = M_2_norm(target_index(2));

    % 7. Net cost definition
    cost = probM1 + probM2;
end
```

To run the optimization workflow, execute the main routine from the MATLAB command window:

```
>> PRESSTO_main('Example_2D')
```

Once the simulation concludes, the `PLOT.m` engine renders a complete validation layout. If you need to re-examine or export the figures later without re-running the solver, execute:

```
>> PLOT('Example_2D')
```

The execution automatically generates the system performance results, which are illustrated and detailed in Fig. 9. These plots display the dynamic progression of the cost function, the binary switching history of the chemical inducers, and the final controlled joint and marginal population distributions.

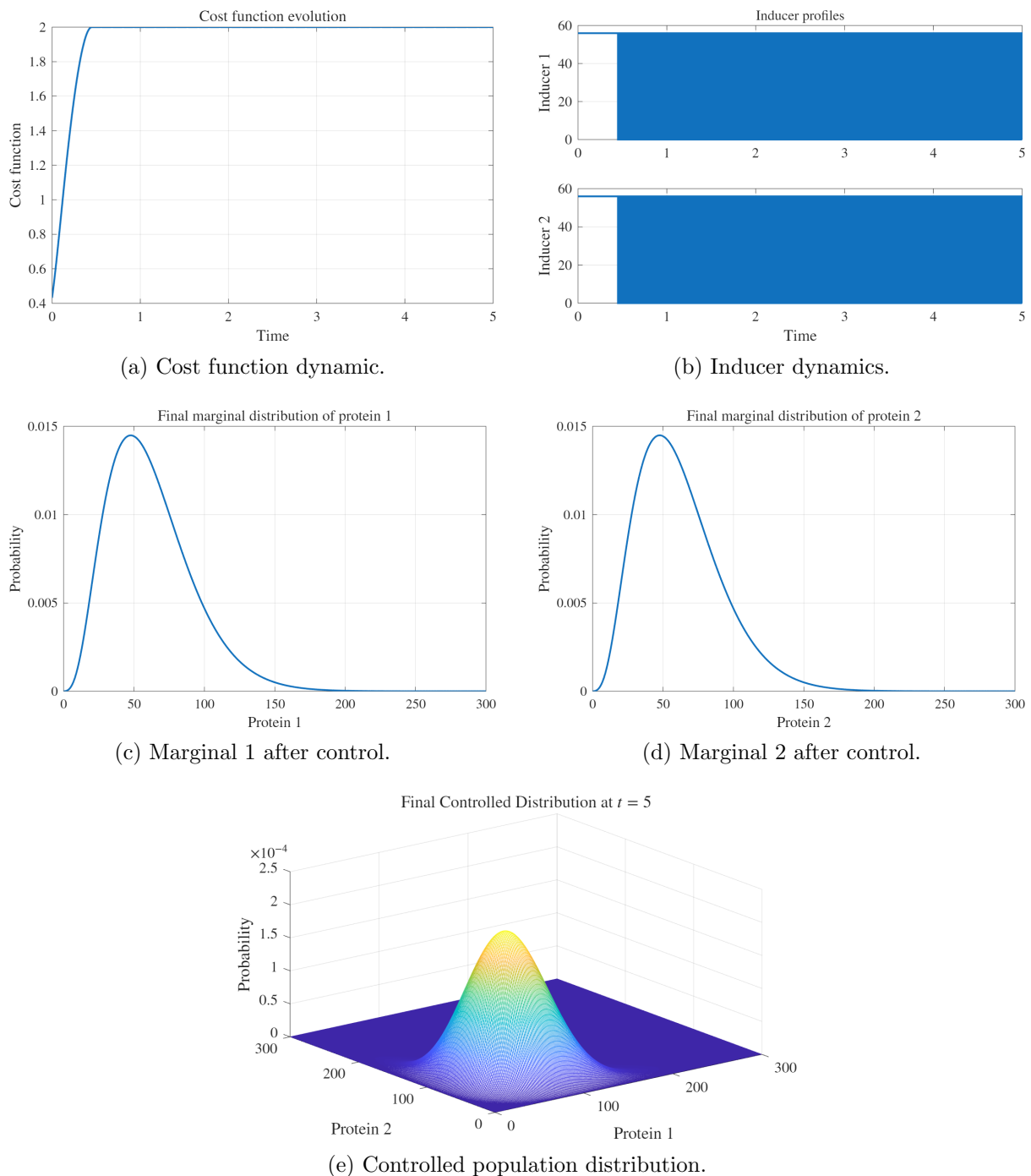


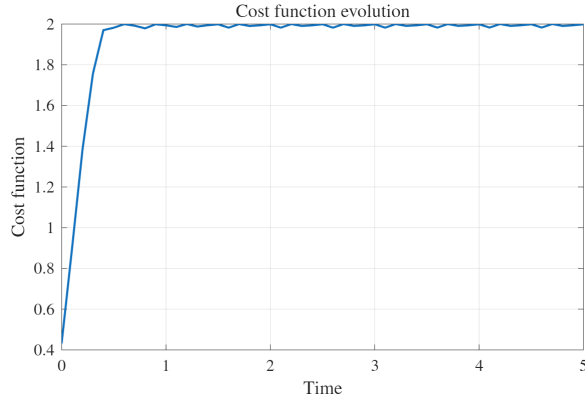
Figure 9: Simulation results for **Example_2D** showing the control performance layout and tracking convergence.

4.2.2 Example_2D_w20: Horizon Expansion ($w = 20$)

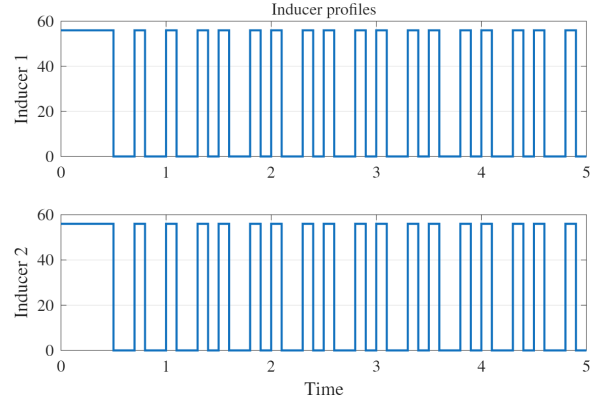
This example evaluates the performance and computational impact of extending the predictive look-ahead window within the PSC algorithm. The biochemical network topology, parametriza-

tion, and cost function remain identical to the baseline configuration described in Section 4.2.1. The unique modification implemented in this script is the expansion of the control horizon parameter within the mesh structure:

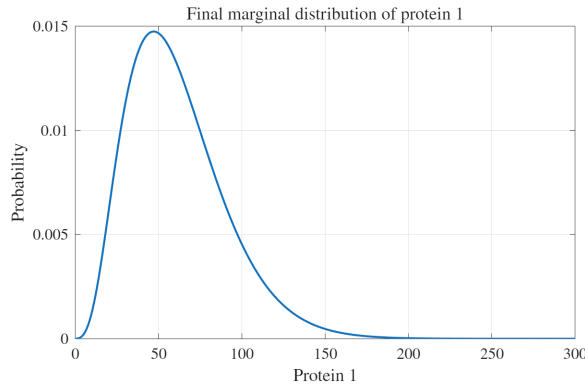
```
meshxt.w = 20; % Control horizon window size
```



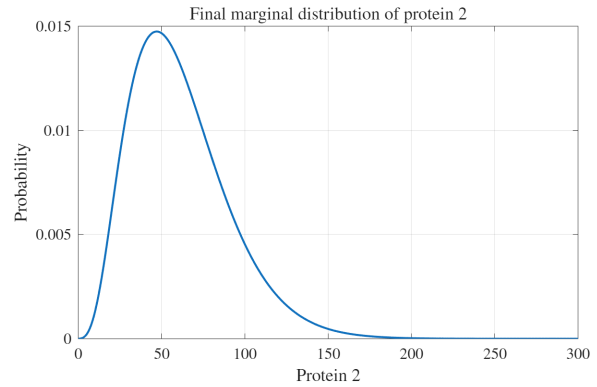
(a) Cost function dynamic.



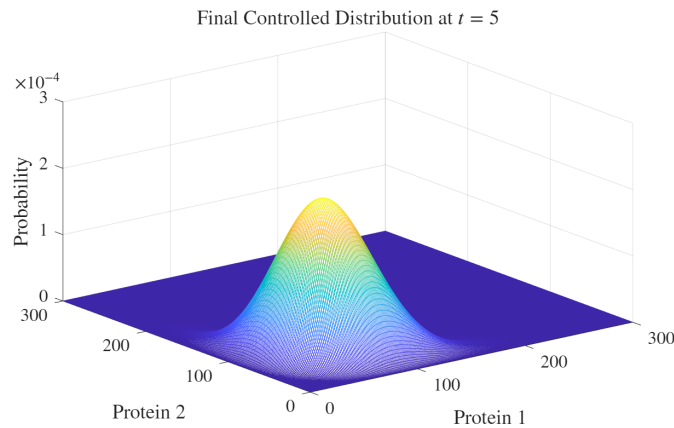
(b) Inducer dynamics.



(c) Marginal 1 after control.



(d) Marginal 2 after control.



(e) Controlled population distribution.

Figure 10: Simulation results for `Example_2D_w20` illustrating the controller performance and tracking response.

By setting $w = 20$, the controller evaluates trajectories over a deeper time horizon, significantly reducing computational overhead by lowering the frequency of optimization steps. This speedup introduces a precision trade-off: while the controller successfully represses network bimodality

by forcing each independent marginal density towards the target $X_i^* = 48$, the steady-state tracking exhibits minor deviations under stochastic noise. Consequently, the cost function values oscillate around the absolute theoretical maximum of $J = 2$ due to the lower sampling frequency.

The script generates the standard set of results plots (Figure 10), tracking the multi-step cost function path, the smoother inducer profiles, and the final controlled joint and marginal distributions. The complete file is available in the **EXPERIMENTS** folder as **Example_2D_w20.m**. To execute it, navigate to the main toolbox directory and run:

```
PRESSTO_main('Example_2D_w20')
```

The resulting trajectory metrics are saved within the **SIMULATIONS** folder. You can directly reload and inspect the figures by calling:

```
PLOT('Example_2D_w20')
```

4.2.3 Example_2D_w100: Large Control Horizon ($w = 100$)

This case pushes the control horizon to a higher value to analyze the algorithm's behavior under a very deep look-ahead window. All biochemical parameters, network topology, and objectives remain identical to the baseline configuration described in 4.2.1.

The only parameter modified in this run is the window size:

```
meshxt.w = 100; % Control horizon window size
```

By setting $w = 100$, the controller evaluates trajectories over a longer time window. This reduces the number of optimization steps, making the simulation much faster. However, as shown in the cost function evolution (see Fig. 11a), this low sampling frequency causes the cost values to oscillate wider around the target compared to the $w = 20$ case. Despite these larger oscillations, the controller still successfully controls the system and maintains a population distribution that largely satisfies the objective function.

The script outputs the standard set of results plots (Figure 11), illustrating the lower cost profile, the highly spaced switching actions, and the final wider distribution. The corresponding source file is located in the **EXPERIMENTS** folder as **Example_2D_w100.m**. It can be executed from the main directory via:

```
PRESSTO_main('Example_2D_w100')
```

The simulation history is exported to the corresponding subfolder in **SIMULATIONS**. To plot again, run:

```
PLOT('Example_2D_w100')
```

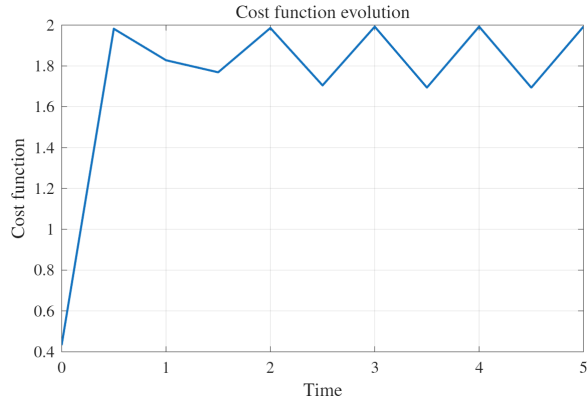
4.2.4 Example_2D_KLdiv: Kullback-Leibler Divergence Minimization

This example demonstrates how to control the system by minimizing the Kullback-Leibler (KL) divergence against a full reference probability distribution. Instead of tracking specific coordinates or marginal peaks, the controller forces the entire 2D population density to match a pre-computed target shape.

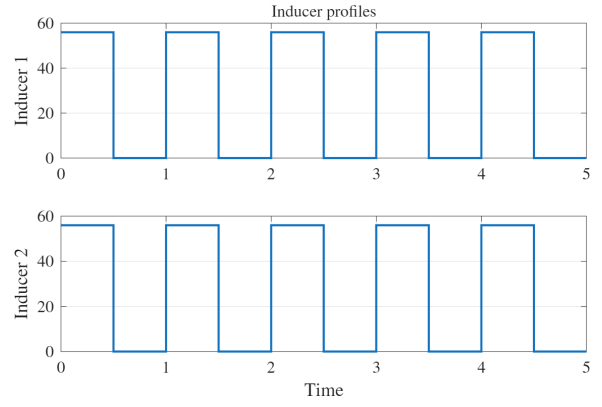
The configuration uses the same network parameters as the baseline case (**Example_2D**), but changes the cost function and the optimization mode:

```
psc.cost_f = @(PX) KL_divergence(PX); % Calls KL-divergence function
psc.opt = 0; % 0 = Minimization mode
```

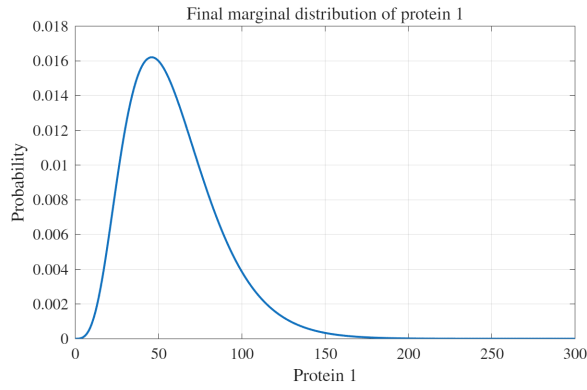
The tracking routine is defined within the script via the following local function:



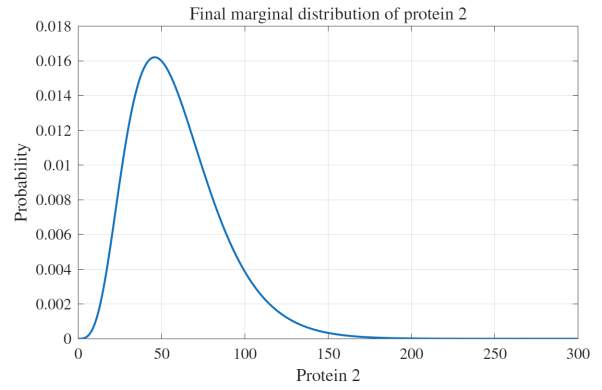
(a) Cost function dynamic.



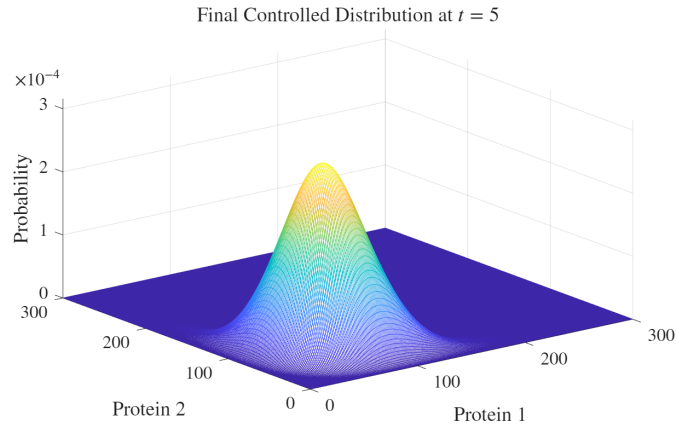
(b) Inducer dynamics.



(c) Marginal 1 after control.



(d) Marginal 2 after control.



(e) Controlled population distribution.

Figure 11: Simulation results for `Example_2D_w100` showing the cost function evolution and control performance.

```
function cost = KL_divergence(PX)
    % 1. Direct load of the .mat file from the absolute path
    PX_target_folder = fullfile(pwd, 'SIMULATIONS', 'Example_2D_KLdiv');
    PX_target = load(fullfile(PX_target_folder, 'PX_target.mat'), 'PX');
    PX;

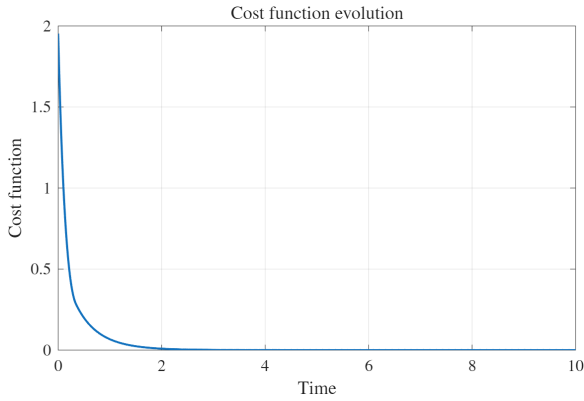
    % 2. Normalization
    P = PX ./ sum(PX(:));
    Q = PX_target ./ sum(PX_target(:));
```

```

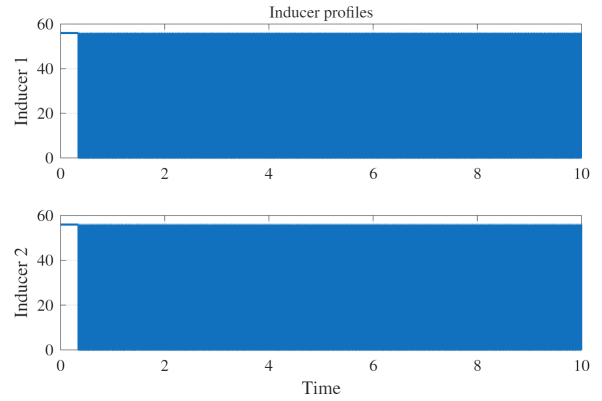
% 3. Infinitesimal correction to safeguard the logarithm against
      zeros
P(P == 0) = eps;    Q(Q == 0) = eps;

% 4. KL Divergence computation: D_KL(P || Q)
kl_matrix = P .* log(P ./ Q);
cost = sum(kl_matrix(:));
end

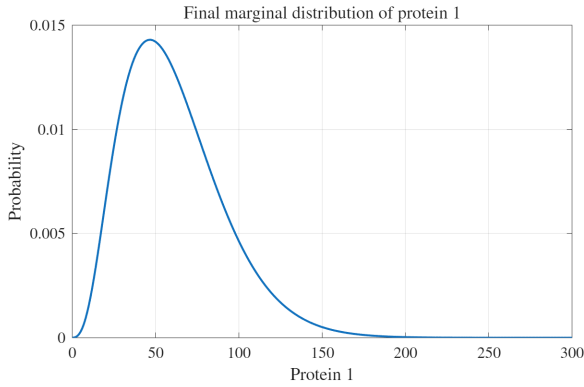
```



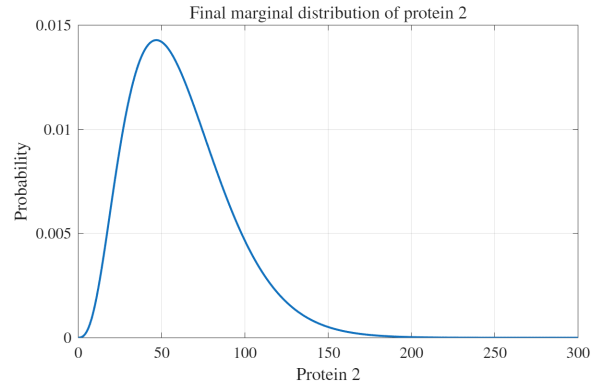
(a) Cost function dynamic.



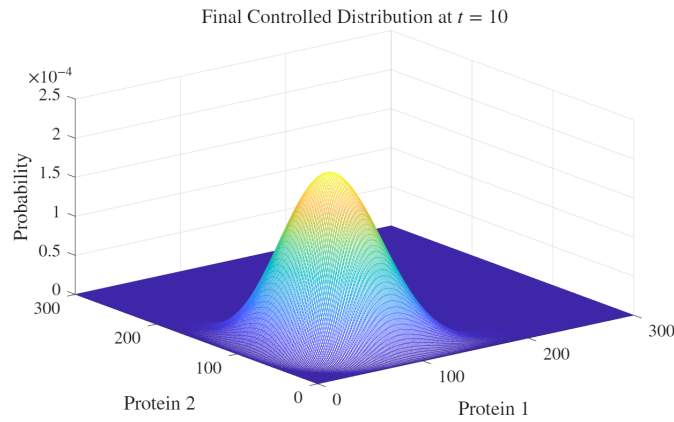
(b) Inducer dynamics.



(c) Marginal 1 after control.



(d) Marginal 2 after control.



(e) Controlled population distribution.

Figure 12: Simulation plots for `Example_2D_KLdiv` illustrating the minimization of the KL divergence.

The target distribution (`PX_target.mat`) is provided within the `SIMULATIONS` folder, inside a subfolder named after the script (`Example_2D_KLdiv`).

In this setup, the algorithm operates in minimization mode (`psc.opt = 0`) to minimize the distance metric:

$$D_{\text{KL}}(P \parallel Q) = \sum_{x_1, x_2} P(x_1, x_2) \log \left(\frac{P(x_1, x_2)}{Q(x_1, x_2)} \right) \quad (36)$$

where P is the current normalized population distribution and Q is the reference target layout. Both arrays are internally normalized and protected with an infinitesimal correction (`eps`) to avoid log-of-zero errors. The controller successfully minimizes this distance, proving that the toolbox can target complex, multi-dimensional distribution profiles rather than just single coordinates.

The script outputs the standard set of results plots (Figure 12), showing the cost function decreasing toward zero, the active switching profiles, and how the final distribution perfectly replicates the target profile. The corresponding source file is located in the `EXPERIMENTS` folder as `Example_2D_KLdiv.m`. It can be executed from the main directory via:

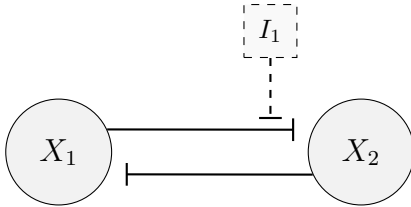
```
PRESSTO_main('Example_2D_KLdiv')
```

The tracking dataset is saved inside the `SIMULATIONS/Example_2D_KLdiv` folder. The divergence profile and distribution plots can be re-rendered via:

```
PLOT('Example_2D_KLdiv')
```

4.2.5 Example_2D_1I: Underactuated Mode Selection

This case study explores an underactuated control scenario where the system is regulated using only a single inducer, as schematically illustrated in Figure 13. The goal is to break the natural bimodal symmetry of the genetic toggle switch, forcing the population density to select the mode where Protein 2 (X_2) is dominant over Protein 1 (X_1).



Parameter	Value
Transcription rates (k_m)	[10, 10]
Translation rates (k_x)	[100, 100]
mRNA degradation rates (γ_m)	[10, 10]
Protein degradation rates (γ_x)	[1, 1]
Leakage rates (ϵ)	[0.1, 0.1]

Figure 13: Genetic toggle switch architecture with a single external inducer (I_1) countering the protein suppression from X_1 to X_2 .

To allow sufficient time for this population shift, the time horizon is extended compared to previous cases:

```
meshxt.Time_mesh = [0 15 3000]; % Extended simulation time
```

The network structure is modified to remove the first inducer, leaving only one inducer function acting on the second gene:

```
% --- Inducer functions (F) ---
grn.F = {};
grn.theta = 0.1;
grn.mu = 2;
grn.F{1} = @(I) 1 ./ (1 + (I ./ grn.theta(1)).^grn.mu(1));
```



```
% --- Regulatory functions (rho) ---
grn.rho = cell(1, grn.n_gene);
grn.K = [40 40];
grn.H = [4 4];
grn.rho{1} = @(x,u) grn.K(1)^grn.H(1) ./ ...
    (grn.K(1)^grn.H(1) + (x{2}.^grn.H(1)));
grn.rho{2} = @(x,u) grn.K(2)^grn.H(2) ./ ...
    (grn.K(2)^grn.H(2) + (x{1}.^grn.H(2)) .* grn.F{1}(u(1)));

% --- Net input functions (c) ---
grn.c = cell(1, grn.n_gene);
grn.c{1} = @(x,u) grn.rho{1}(x,u) + grn.eps(1)*(1-grn.rho{1}(x,u));
grn.c{2} = @(x,u) grn.rho{2}(x,u) + grn.eps(2)*(1-grn.rho{2}(x,u));
```

To drive the population toward the target state, the cost function is designed to maximize the probability mass at the dominant coordinates [6, 96], while simultaneously minimizing the mass at the opposite bimodal peak [96, 6] using maximization mode (`psc.opt = 1`):

```
% Cost function
target_point = [6 96];
psc.cost_f = @(PX) PX(target_point(1),target_point(2))-PX(target_point
    (2),target_point(1));
psc.opt = 1; % Maximization mode
```

The controller successfully unbalances the system using the single input line. The probability mass shifts cleanly into the chosen state, showing that PRESSTO can handle asymmetric underactuated designs.

The script outputs the standard set of results plots (Figure 14). These figures show the cost function convergence, the single active inducer profile, and the final single-mode distribution. The file is located in the `EXPERIMENTS` folder as `Example_2D_1I.m`. Run it from the main folder with:

```
PRESSTO_main('Example_2D_1I')
```

All single-input control logs are archived upon completion. To view the asymmetric mode selection plots again, execute:

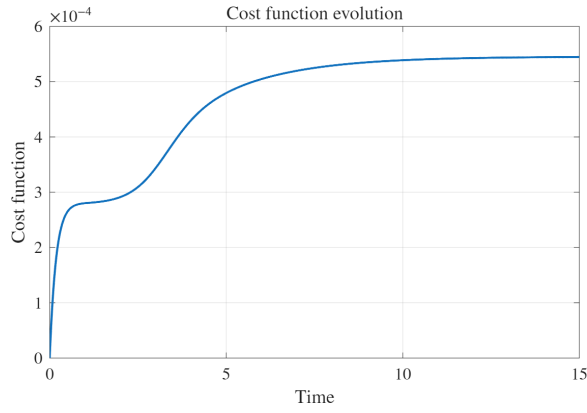
```
PLOT('Example_2D_1I')
```

4.3 Case 3: Three-gene regulatory systems

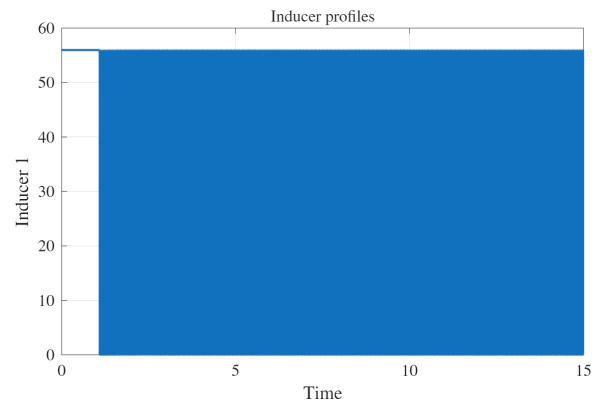
This section extends the application of the toolbox to higher-dimensional spaces by analyzing a gene regulatory network composed of three interacting nodes. Moving from two to three dimensions increases the computational complexity of the probability grid, but it also allows the study of more intricate network topologies and control challenges. The following case study illustrates how the framework handles structural asymmetries across multiple genes to enforce a balanced population profile under a topology where each repression pathway is regulated by a dedicated external inducer (Figure 15).

4.3.1 Example_3D: Symmetry Forcing

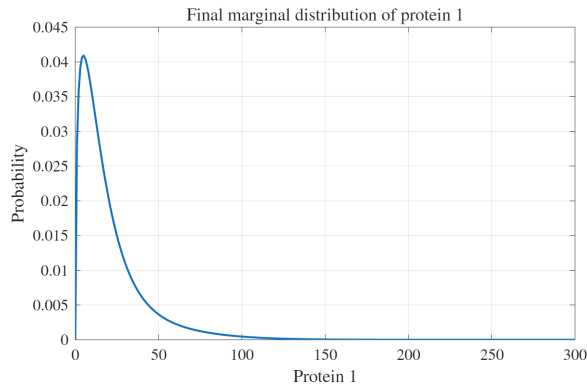
This case study expands the application of the Predictive Switching Control (PSC) framework to a three-dimensional genetic regulatory network (GRN) consisting of three interacting genes. In its natural, uncontrolled steady state, this system is inherently asymmetric due to variations in transcription, translation, and degradation rates across the three nodes. This structural asymmetry causes the uncontrolled population distribution to stabilize around a specific asymmetric state with average protein copy numbers at $X_1 = 284$, $X_2 = 236$ and $X_3 = 204$.



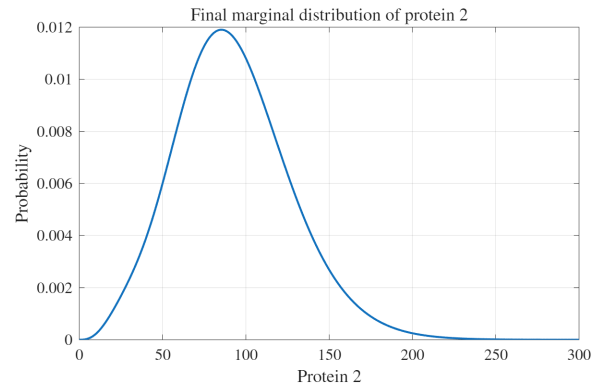
(a) Cost function dynamic.



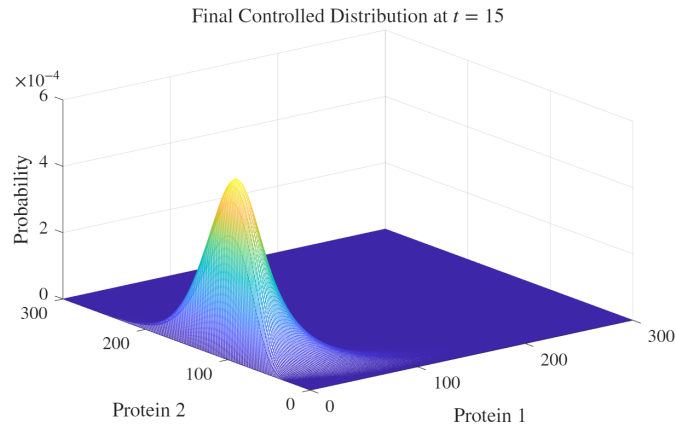
(b) Inducer dynamics.



(c) Marginal 1 after control.

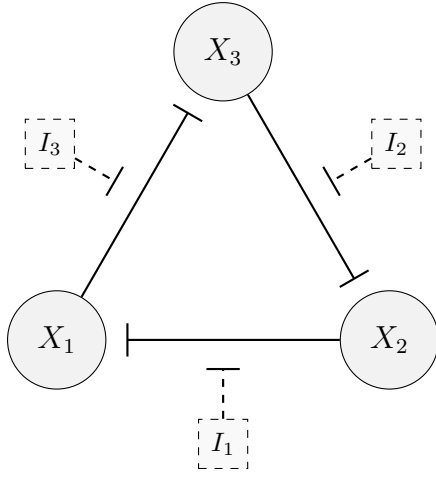


(d) Marginal 2 after control.



(e) Controlled population distribution.

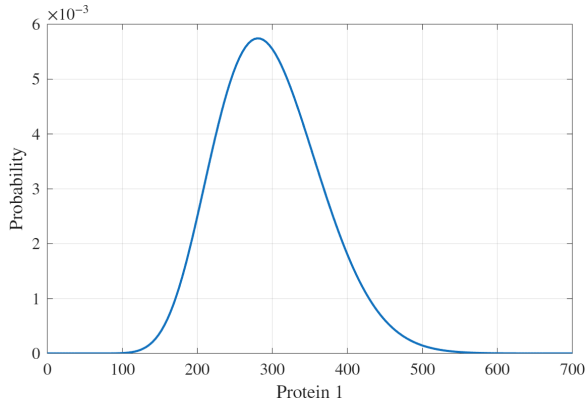
Figure 14: Dynamic performance of the `Example_2D_1I` network topology. The plots illustrate how the predictive switching controller modulates the single inducer profile to shift the probability mass toward Protein 2 dominance.



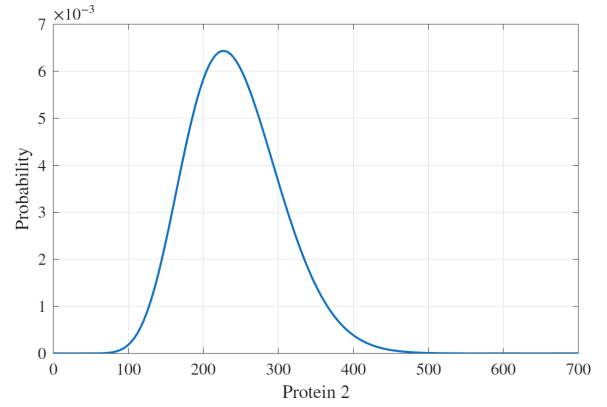
Parameter	Value
Transcription rates (k_m)	[121, 96, 108]
Translation rates (k_x)	[75, 60, 110]
mRNA degradation (g_m)	[15, 10, 20]
Protein degradation (g_x)	[1, 1.2, 1.15]
Leakage rates (ε)	[0.12, 0.15, 0.13]

Figure 15: 3D repressilator network architecture configured in a clockwise feedback loop ($X_1 \dashv X_3 \dashv X_2 \dashv X_1$). Each loop interaction is modulated by a specific external inducer (I_1, I_2, I_3).

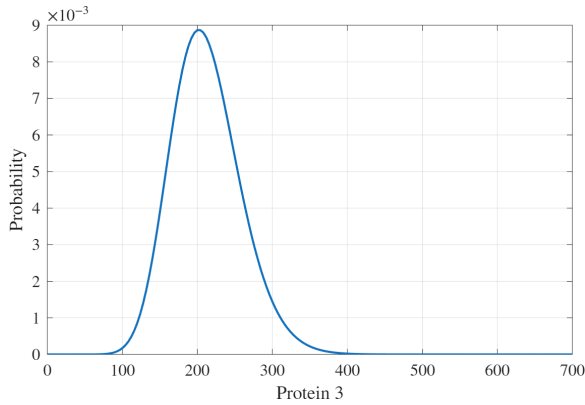
Figure 16 presents the pre-control status, showing the three independent marginal distributions and the joint 3D population density.



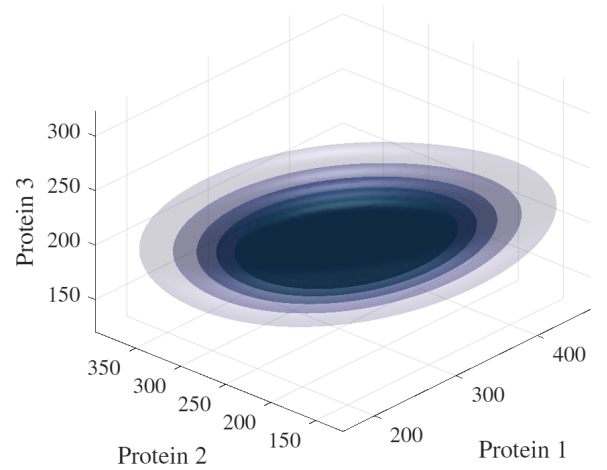
(a) Marginal 1



(b) Marginal 2



(c) Marginal 3



(d) Steady-state distribution

Figure 16: Uncontrolled steady-state distribution for **Example_3D** showing the natural asymmetric marginals and 3D joint distribution.

The control objective here is to force the system away from this asymmetric profile and stabilize

the cell population around a symmetric intermediate state. Specifically, the controller aims to balance the expression levels so that all three proteins are maintained at a similar concentration.

Script 3: Complete configuration for the 3D asymmetric network case study (Example_3D.m).

```

1 function config = Example_3D()
2 %% ===== OUTPUT FOLDER =====
3 config.name = mfilename;
4
5 %% ===== GENE REGULATORY NETWORK =====
6 % Number of genes
7 grn.n_gene = 3;
8
9 % --- Rates ---
10 grn.k_m = [121 96 108]; % Transcription rates (k_m)
11 grn.k_x = [75 60 110]; % Translation rates (k_x)
12 grn.g_m = [15 10 20]; % mRNA degradation (g_m)
13 grn.g_x = [1 1.2 1.15]; % Protein degradation (g_x)
14 grn.eps = [0.12 0.15 0.13]; % Leakage rates (eps)
15
16 % --- Inducer functions (F) ---
17 grn.F = cell(1, grn.n_gene);
18 grn.theta = [1.4 3.2 0.89];
19 grn.mu = [3 4 2];
20 grn.F{1} = @(I) 1 ./ (1 + (I ./ grn.theta(1)).^grn.mu(1));
21 grn.F{2} = @(I) 1 ./ (1 + (I ./ grn.theta(2)).^grn.mu(2));
22 grn.F{3} = @(I) 1 ./ (1 + (I ./ grn.theta(3)).^grn.mu(3));
23
24 % --- Regulatory functions (rho) ---
25 grn.rho = cell(1, grn.n_gene);
26 grn.K = [210 202 198];
27 grn.H = [3 4 2];
28 grn.rho{1} = @(x,I) grn.K(1)^grn.H(1) ./ ...
29     (grn.K(1)^grn.H(1) + x{2}.^grn.H(1) .* grn.F{1}(I(1)));
30 grn.rho{2} = @(x,I) grn.K(2)^grn.H(2) ./ ...
31     (grn.K(2)^grn.H(2) + x{3}.^grn.H(2) .* grn.F{2}(I(2)));
32 grn.rho{3} = @(x,I) grn.K(3)^grn.H(3) ./ ...
33     (grn.K(3)^grn.H(3) + x{1}.^grn.mu_rho(3) .* grn.F{3}(I(3)));
34
35 % --- Net input functions (c) ---
36 grn.c = cell(1, grn.n_gene);
37 grn.c{1} = @(x,I) grn.rho{1}(x,I) + grn.eps(1).*(1-grn.rho{1}(x,I));
38 grn.c{2} = @(x,I) grn.rho{2}(x,I) + grn.eps(2).*(1-grn.rho{2}(x,I));
39 grn.c{3} = @(x,I) grn.rho{3}(x,I) + grn.eps(3).*(1-grn.rho{3}(x,I));
40
41 config.grn = grn;
42
43 %% ===== MESH =====
44 % Spatial grid (protein counts) and temporal simulation mesh
45 meshxt.Prot_mesh = [0 700 175; 0 700 175; 0 700 175]; % [min, max,
46     number of spatial intervals] per gene
47 meshxt.Time_mesh = [0 0.5 100]; % [t0, t_end, number of time intervals]
48 meshxt.w = 1; % Control horizon window size
49
50 config.meshxt = meshxt;
51
52 %% ===== INITIAL CONDITION =====
53 ic.mode = 0; % 0 = Analytical Gaussian, 1 = Load external .mat
54     distribution

```

```

53 switch ic.mode
54     case 0
55         ic.mean = [260 260 260];           % Initial population mean
56         ic.var  = [100 100 100];           % Initial population variance
57     end
58
59 config.ic = ic;
60
61 %% ===== PSC PARAMETERS =====
62 % Activation level = 1 - alpha_i (one per inducer)
63 psc.alpha = [0.005 0.01 0.015];
64
65 % Cost function
66 target_point = [260, 260, 260];
67 dx = (meshxt.Prot_mesh(:,2)' - meshxt.Prot_mesh(:,1)') ./ meshxt.
        Prot_mesh(:,3)';
68 target_index = round((target_point - meshxt.Prot_mesh(:,1)') ./ dx) +
        1;
69 psc.cost_f = @(PX) PX(target_index(1), target_index(2), target_index(3)
        ) / max(PX(:));
70
71 % Optimization mode: 0 = Minimize, 1 = Maximize
72 psc.opt = 1;
73
74 config.psc = psc;
75
76 end

```

The operational parameters that characterize the PSC setup for this 3D case study are detailed below:

- **Control Horizon (meshxt.w):** Configured to execute optimizations at every step using a single-step look-ahead window.

```
meshxt.w = 1;           % Control horizon window size
```

- **Initial Condition (ic.mode, ic.mean, ic.var):** Initialized from an analytical Gaussian distribution centered exactly at the target state coordinates $X_i^* = 260$, to evaluate the controller's ability to maintain and stabilize the population around this symmetric target profile (ic.mode=0).

```
ic.mode = 0;           % 0 = Analytical Gaussian
ic.mean = [260 260 260]; % Initial population mean
ic.var  = [100 100 100]; % Initial population variance
```

- **Boundary Suppression Threshold (psc.alpha):** Establishes the multi-variable vector $\alpha = [\alpha_1, \alpha_2, \alpha_3]$ used to calculate the saturation value κ_i of each independent inducer.

```
psc.alpha = [0.005 0.01 0.015]; % Suppression thresholds per gene
```

- **Cost Function (psc.cost_f, psc.opt):** Formulated to track and maximize the probability density at the symmetric target coordinates, normalized by the maximum value of the distribution ($\max(PX(:))$). This setup ensures that the cost function reaches its peak value of 1 if the target point matches the absolute mode of the population profile. The index in the 3D grid is automatically computed using the mesh step size dx . Setting $psc.opt = 1$ configures the algorithm to run in maximization mode:

```

target_point = [260, 260, 260];
dx = (meshxt.Prot_mesh(:,2)' - meshxt.Prot_mesh(:,1)') ./ meshxt.
    Prot_mesh(:,3)';
target_index = round((target_point - meshxt.Prot_mesh(:,1)') ./ dx)
    + 1;
psc.cost_f = @(PX) PX(target_index(1), target_index(2),
    target_index(3)) / max(PX(:));
psc.opt = 1; % 1 = Maximization mode

```

Despite the structural asymmetry of the underlying biological network, the controller successfully coordinates the three inducer profiles to balance the expression levels. The final population distribution is effectively reshaped and confined around the symmetric target.

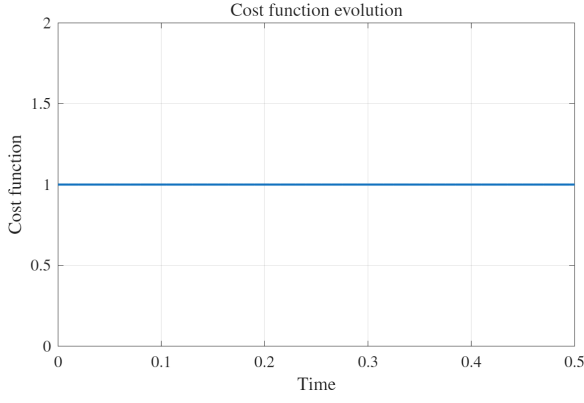
The post-control performance is illustrated in Figure 17. This standard validation figure captures the cost function evolution, the three active inducer profiles over time, the final balanced marginal distributions, and the controlled 3D joint distribution profile.

The complete implementation is provided in the script below. Due to the increased dimensionality of both the GRN grid and the PSC optimization space, the computational load is significantly higher than in the 2D cases; consequently, the execution might take longer and require several minutes to complete. The file is located in the **EXPERIMENTS** folder as **Example_3D.m**, and can be executed by running:

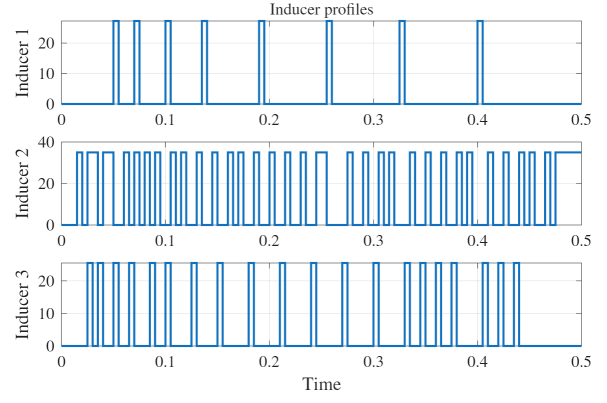
```
PRESST0_main('Example_3D')
```

The full 3D grid data and coordinate histories are saved in the **SIMULATIONS** workspace. To rebuild the multi-panel marginal and joint density figures, use:

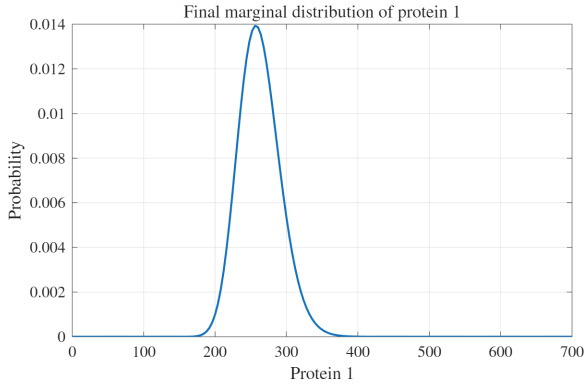
```
PLOT('Example_3D')
```



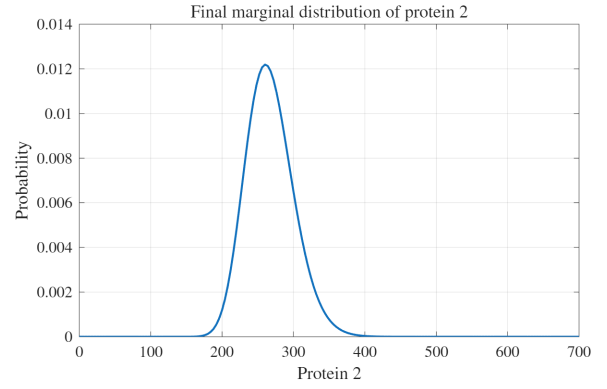
(a) Cost function dynamic.



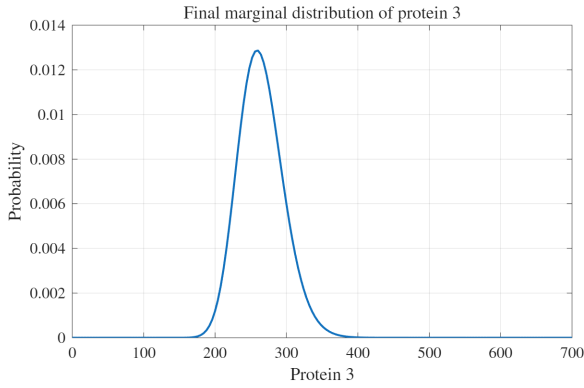
(b) Inducer dynamics.



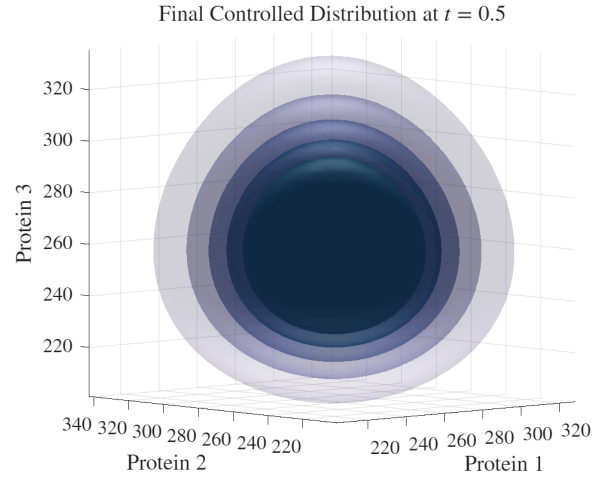
(c) Marginal 1 after control.



(d) Marginal 2 after control.



(e) Marginal 3 after control.



(f) Controlled population distribution.

Figure 17: Controlled simulation results for `Example_3D` capturing the cost trajectory, inducer profiles, and symmetric target distribution tracking.

References

Christian Fernández, Manuel Pájaro, Gábor Szederkényi, and Irene Otero-Muras. Predictive-Switching Control of Stochastic Gene Regulatory Networks: A Contractive PIDE Framework, 2026.

Manuel Pájaro, Antonio A. Alonso, Irene Otero-Muras, and Carlos Vázquez. Stochastic model-

ing and numerical simulation of gene regulatory networks with protein bursting. *Journal of Theoretical Biology*, 421:51–70, 2017.

Manuel Pájaro, Irene Otero-Muras, Carlos Vázquez, and Antonio A. Alonso. SELANSI: SemiLagrangian Numerical Simulation of Gene Regulatory Networks. *Bioinformatics*, 34(5):893–895, 2018.